

RAG AGAINST THE MACHINE — 42 PROJECT SERIES

VOLUME 3 OF 3

Evaluation, Bonuses & Defense

Make it score, make it fast, make it unbreakable — then defend every line of it

Covers: the moulinette in depth • recall triage (FILE/SPAN/RANK misses) • tuning chunk size, k1/b, boosts • performance engineering against both budgets • the hostile-review gauntlet • all five bonuses implemented (semantic, hybrid RRF, incremental, caching, HTTP API) • the README section by section • the defense and recode drills • the final checklist

Companion study guide — subject version 2.0 — not an official 42 document

Contents

The Moulinette, In Depth	5
The grading command and its stages	5
Reading the output like a diagnostician	5
A validation self-check before every moulinette run	6
Recall Triage: Debugging a Failing Score	7
The three failure classes and their cures	7
Tuning the Retriever	9
Knob 1 - chunk size	9
Knob 2 and 3 - k1 and b	9
Knob 4 - stemming and stoplist	9
Knob 5 - field boosts and per-dataset strategy	10
What a healthy endpoint looks like	10
Performance Engineering	11
Measure first: the profile	11
The 90-second budget, dissected	11
Memory	12
Hardening Against a Hostile Review	13
The mutation drill	13
Bonus 1: Semantic Embeddings	15
Design	15
Integration and cost accounting	16
Bonus 2: Hybrid Retrieval	17
Bonus 3: Incremental Indexing	19
Bonus 4: Caching	21
Bonus 5: A Local HTTP API	23
A Complete Tuning Case Study	25
Iteration 0 — the naive baseline	25
Iteration 1 — fix discovery filters	25
Iteration 2 — identifier splitting in the analyzer	25
Iteration 3 — structural chunking	26
Iteration 4 — path-term boosting	26
Iteration 5 — k1/b grid	26

Iteration 6 — per-file dedup in top-k	26
What the campaign teaches	26
Question Post-Mortems	28
1 — The paraphrase (FILE-MISS, docs)	28
2 — The verbatim identifier (should be easy; missed by over-normalization)	28
3 — The split table (SPAN-MISS, docs)	28
4 — The scattered class (SPAN-MISS/RANK-MISS, code)	28
5 — The crowd-out (RANK-MISS, multi-source)	29
6 — The stopworded signal (FILE-MISS, code)	29
7 — The wrong-prefix ghost (systemic zero)	29
8 — The question your corpus genuinely cannot answer	29
Beyond the Subject: The Next Three Techniques	31
Re-ranking with a cross-encoder	31
Query expansion	31
Structured filtering and routing	31
Defense Drill II: Thirty Questions on Practice	33
Build and pipeline	33
Scoring and tuning	33
Generation and bonuses	34
The README, Section by Section	35
A skeleton that satisfies the list	35
The AI-usage paragraph	36
The Defense	37
How the session typically flows	37
Answers you should have loaded, verbatim-ready	37
Conduct under fire	38
The Final Checklist	39
Compliance	39
Scoring	39
Robustness	39
Story	39
Closing: What This Project Actually Taught You	41
Appendix: The Complete Parameter Reference	42
Indexing	42

Scoring	42
Generation	43
Bonuses	43
Appendix: Operations on Campus Machines	45
Where the gigabytes actually go	45
Pre-downloading and offline resilience	45
Shared-machine etiquette that doubles as engineering	46
Appendix: An Annotated Defense Transcript	47
Minutes 0-8: setup and the automated run	47
Minutes 8-18: exploration	47
Minutes 18-32: theory and code reading	47
Minutes 32-40: recode and bonuses	48
What made this defense strong	48
The one-page crib sheet	49
Series topic index	49

CHAPTER 1

The Moulinette, In Depth

Know your judge. Everything the grader checks, in the order it checks it — and what each message really means.

This final volume assumes the Volume 2 build is complete and green. It has one goal: convert a *working* system into a *scoring, robust, defensible* one. We start with the instrument that defines success. The moulinette ships in the repository as a platform-named binary (moulinette-ubuntu / moulinette-fedora); rename it to `./moulinette`, make it executable, and never import or call it from your own code — your `evaluate` command re-implements the metric from the spec (Volume 2 ch. 12), it does not wrap the binary.

The grading command and its stages

Argument order: YOUR results first, ground truth second

```
./moulinette evaluate_student_search_results \
  <student_results_path> <dataset_path> \
  --k 10 --max_context_length 2000
```

Internally the run has three stages, and each failure mode maps to one:

1. **Validation.** The results file must parse as `StudentSearchResults`; every retrieved source must have an existing schema, spans with last > first, and width at most `max_context_length` (2000). One violation anywhere prints 'Student data is valid: False' and the run is over — recall is not even computed. This all-or-nothing rule is why Volume 2 asserts the invariant at chunk construction AND at serialization.
2. **Matching.** For each question, each ground-truth source is checked against your top-k: same verbatim `file_path` and interval IoU at least 0.05 (the geometry drilled in Volume 1 ch. 11).
3. **Reporting.** Macro-averaged `Recall@1/3/5/10` are printed; the tool's boolean verdict is `Recall@5` against the dataset's threshold — 0.80 for docs, 0.50 for code.

Reading the output like a diagnostician

Observation	Diagnosis	Where to act
valid: False	schema drift or an oversized span	Vol 2 ch. 2/5; find the offending entry with a 20-line script
valid: True, all recalls 0.000	path prefix mismatch (verbatim compare)	store <code>data/raw/vllm-0.10.1/...</code> from discovery
recall@1 decent, @5 barely higher	top-1 fine but little diversity below it	chunk dedup, diversity, per-file caps (ch. 3)

Observation	Diagnosis	Where to act
@5 close to @10, both low	right ordering, wrong candidates: chunking or analyzer holes	chunker boundaries, file coverage
@10 much higher than @5	candidates found but ordered low	scoring: k1/b, boosts, stemming (ch. 3)
docs passes, code fails	identifier handling or code chunk boundaries	analyzer splitting; AST chunker descent
code passes, docs fails	headings not exploited, stopword noise	markdown chunker; short stoplist; boosts

Commit this table to memory — it converts a bare number into a work order. The recall staircase (four numbers, one shape) is the most information-dense diagnostic you own; Volume 1 ch. 11 explains why each gap means what it means.

A validation self-check before every moulinette run

Ten lines that make 'valid: False' a thing that happens to other people

```
from pathlib import Path
from src.models import StudentSearchResults, load_model

res = load_model(Path("data/output/.../dataset_docs_public.json"),
                  StudentSearchResults)
for r in res.search_results:
    for s in r.retrieved_sources:
        width = s.last_character_index - s.first_character_index
        assert 0 < width <= 2000, (r.question_id, s)
        assert s.file_path.startswith("data/raw/"), s.file_path
print("pre-flight ok:", len(res.search_results), "questions")
```

CHAPTER 2

Recall Triage: Debugging a Failing Score

A low number is not a diagnosis. Break it into named failures, fix the biggest class, re-measure, repeat.

Suppose docs recall@5 sits at 0.61 against the 0.80 bar. The wrong move is 'try different parameters and hope'. The right move is to find out, question by question, *which ground-truth sources were missed and why*. One script does it:

The triage script: every miss becomes one of three named failures

```
"""triage.py - classify every miss (dev tool, not part of src)."""
from pathlib import Path

from src.evaluation import _iou, IOU_THRESHOLD
from src.models import (
    AnsweredQuestion, RagDataset, StudentSearchResults, load_model,
)

results = load_model(Path("data/output/search_results/"
                          "UnansweredQuestions/dataset_docs_public.json"),
                    StudentSearchResults)
dataset = load_model(Path("data/datasets/AnsweredQuestions/"
                          "dataset_docs_public.json"), RagDataset)
truth = {q.question_id: q for q in dataset.rag_questions
         if isinstance(q, AnsweredQuestion)}

for res in results.search_results:
    gt = truth.get(res.question_id)
    if gt is None:
        continue
    top5 = res.retrieved_sources[:5]
    for g in gt.sources:
        if any(_iou(r, g) >= IOU_THRESHOLD for r in top5):
            continue # found - not interesting
        same_file = [r for r in res.retrieved_sources
                     if r.file_path == g.file_path]
        if not same_file:
            kind = "FILE-MISS" # right file never retrieved
        elif any(_iou(r, g) >= IOU_THRESHOLD for r in same_file):
            kind = "RANK-MISS" # found, but below top-5
        else:
            kind = "SPAN-MISS" # right file, wrong region
        print(f"{kind} {res.question[:60]!r}\n"
              f"      wanted {g.file_path} "
              f"({g.first_character_index}-{g.last_character_index})")
```

The three failure classes and their cures

Class	Meaning	Typical causes	Cures
FILE-MISS	the right file appears nowhere in top-k	vocabulary mismatch; file never indexed; query terms diluted	analyzer improvements (splitting, stemming); check discovery coverage; path-term boosting; (bonus) hybrid retrieval
SPAN-MISS	right file, but your chunks there don't overlap enough	chunk boundaries cut the target region; chunks too small	structure-aware boundaries; larger chunks toward the 2000 ceiling; overlap on hard splits
RANK-MISS	a matching chunk exists but ranks 6-10 or lower	scoring order; near-duplicate chunks crowding top-5	tune k1/b; deduplicate per-file; boosts; better IDF via cleaner indexing

Count the classes before touching anything. Fifty FILE-MISSES and three RANK-MISSES means tuning k1 is procrastination; the analyzer or coverage is the job. The distribution also picks your reading: FILE-MISS sends you to Volume 1 ch. 8, SPAN-MISS to ch. 9, RANK-MISS to ch. 7. Re-run triage after every single change — classes shift, and a cure for one class can mildly worsen another; the macro number alone would hide that exchange.

Note — Keep a lab notebook (a markdown file in your repo is fine): date, change, recall@5 docs, recall@5 code, triage counts. Ten lines a day. It becomes the 'Performance analysis' and 'Challenges faced' sections of your README almost verbatim — and it is exactly the artifact that impresses at a defense.

CHAPTER 3

Tuning the Retriever

Chunk size, k1, b, stemming, boosts: five knobs, one harness, measured — never guessed.

All tuning is the same loop: change one thing, re-index if it touched indexing, run `search_dataset`, run YOUR `evaluate` (fast), log the numbers. Automate the loop before turning any knob:

The measurement harness: `./sweep.sh 1800` prints four recall lines

```
#!/usr/bin/env bash
# sweep.sh - one measurement per invocation
set -e
uv run python -m src index --max_chunk_size "$1"
for ds in docs code; do
  uv run python -m src search_dataset \
    --dataset_path data/datasets/UnansweredQuestions/dataset_${ds}_public.json \
    --k 10 \
    --save_directory data/output/search_results/UnansweredQuestions
  uv run python -m src evaluate \
    --student_search_results_path \
      data/output/search_results/UnansweredQuestions/dataset_${ds}_public.json \
    --dataset_path data/datasets/AnsweredQuestions/dataset_${ds}_public.json
done
```

Knob 1 - chunk size

Sweep `max_chunk_size` over {800, 1200, 1600, 2000}. Expectations from the geometry (Volume 1 ch. 9 and 11): larger chunks raise the probability of containing a ground-truth span (IoU bar is only 0.05) but dilute term statistics slightly; smaller chunks sharpen scoring but multiply SPAN-MISSes. On most implementations the curve rises toward the ceiling with a knee around 1500-2000 — but run YOUR sweep: the subject explicitly asks you to 'report the effect on your recall@k', and the README's performance-analysis section wants the table. Note that chunk size interacts with the chunker's merge logic; re-run triage at each point, not just the headline number.

Knob 2 and 3 - k1 and b

Grid {0.9, 1.2, 1.5, 2.0} x {0.3, 0.5, 0.75, 0.9} costs sixteen index-free runs (BM25 parameters live in scoring, so you do NOT re-index — expose them as retriever constants or CLI options). Typical movement is one to three recall points; the docs and code datasets often prefer different corners (prose rewards standard 1.5/0.75; code often likes lower k1, per the saturation argument in Volume 1 ch. 7). If the two datasets disagree strongly, that is your cue for knob 5.

Knob 4 - stemming and stoplist

Add Snowball stemming behind a flag in `analyze()`; re-index; measure. Keep it only if the delta is positive on docs without hurting code (identifier parts should NOT be stemmed — stem only tokens that were not produced by identifier splitting, or accept the wash and report it). Similarly try the

stoplist at 0 words and at your ~30: the measurement costs two runs and settles a debate that opinions cannot.

Knob 5 - field boosts and per-dataset strategy

- **Path-term boosting:** append the analyzed terms of the file path to every chunk's term list at indexing time (cheap BM25F, Volume 1 ch. 7). A question mentioning 'lora' now favours everything under vllm/lora/ and docs/features/lora.md. Implementation: in `_add_chunk`, `terms += analyze(chunk.file_path.replace('/', ' '))`.
- **Heading boosting** for Markdown: duplicate the section heading's terms once per chunk — headings are titles, titles are queries.
- **Extension-aware scoring:** docs questions overwhelmingly resolve to .md files, code questions to .py (chapter 13 of Volume 2 showed you the measurement). A small additive bonus for .md chunks when evaluating the docs dataset (and .py for code) is legitimate IF you expose it honestly (a CLI flag or auto-detection by dataset filename) and can defend it. The cleaner variant: keep one index but report per-extension recall in triage, and fix the real losses first.

Watch out — Tune against the PUBLIC datasets, then stop. Fitting question-specific hacks (matching exact question strings, hardcoding paths) is overfitting the visible set and will both fail the private set and torpedo the defense when someone reads the code. Every boost must be a general principle you can argue from theory.

What a healthy endpoint looks like

Metric	Passing	Comfortable	Excellent
docs recall@5	0.80	0.85+	0.90+
code recall@5	0.50	0.58+	0.65+
docs recall@10 - recall@5	any	< 0.08	< 0.05
indexing time	< 5 min	< 3 min	< 90 s
200-question search	< 90 s	< 15 s	< 5 s

Aim for 'comfortable': the private datasets are drawn from the same distribution but not identical, and a margin absorbs the shift. If you are at 0.80 exactly on the public docs set, treat yourself as failing and go another triage round.

CHAPTER 4

Performance Engineering

Two hard budgets — five minutes to index, ninety seconds for two hundred questions — and how to spend them well.

Measure first: the profile

Profile before optimizing; the top-5 cumulative lines name the problem

```
# Where does indexing time go?
uv run python -m cProfile -s cumulative -m src index 2>&1 | head -25

# Wall-clock the pieces
time uv run python -m src index --max_chunk_size 2000
time uv run python -m src search_dataset --dataset_path \
  data/datasets/UnansweredQuestions/dataset_docs_public.json \
  --k 10 --save_directory /tmp/perfcheck
```

On a healthy implementation the indexing profile is dominated by `analyze()` regex work and AST parsing — both linear and both already fast enough. The pathologies that actually blow the 5-minute budget, in the order students hit them:

1. **Indexing what should be filtered:** giant generated files, binaries, tests you decided to skip but didn't. Symptom: `n_chunks` far above expectation. Fix: `chapter-3-of-Volume-2` filters; print per-kind counts.
2. **Quadratic string building:** accumulating chunk text with `+=` in a loop. Fix: slice from the file text (Volume 2 does — `text[start:end]` is $O(\text{len})$) and join lists.
3. **Re-analyzing per query at batch time:** reloading the index or the model per question. Fix: load once per process (Volume 2's CLI does); if you see per-question index loads in a profile, the bug is in your command wiring.
4. **Pickling per chunk** instead of once: `save_index` must run once, at the end.

The 90-second budget, dissected

Volume 2's measurement showed 200 BM25 queries complete in seconds — the budget is effectively 30x oversubscribed. Spend the surplus deliberately: it is your headroom for the bonuses (dense retrieval adds an embedding forward pass per query — tens of milliseconds on CPU for MiniLM; hybrid fusion adds microseconds) and for any re-ranking experiments. The one way students lose the budget is accidental: importing `torch/transformers` at module top-level in a file the search path touches, adding a 5-15 s import to every invocation and, worse, per-question model loads in `answer_dataset`. Volume 2's deferred imports and module-level cache are the pattern; verify with `time uv run python -m src search 'test'` that a plain search never pays the torch tax.

Memory

The index holds every chunk's text in RAM — tens of MB, fine. The embedding matrix (bonus 1) adds $n_chunks \times 384 \text{ float32} \sim 40\text{-}60 \text{ MB}$, fine. Qwen3-0.6B in float32 is $\sim 2.4 \text{ GB}$ RAM (bfloat16 halves it; `torch_dtype='auto'` picks sensibly). The binding constraint on a campus machine is usually disk for the HF cache, not RAM — the subject's closing warning about gigabytes of deep-learning stack is aimed exactly here. Pre-download on defense-day hardware.

Note — Report your measured numbers (index time, batch time, peak memory via `/usr/bin/time -v`) in the README performance section. Concrete figures signal engineering maturity and preempt the evaluator's stopwatch.

CHAPTER 5

Hardening Against a Hostile Review

The evaluator's job is to push your system in odd directions. Push it yourself first.

Volume 2 ch. 10 built the error boundary and walked the subject's listed edge cases. This chapter is the adversarial extension: the inputs an inventive reviewer actually tries, beyond the listed minimum. Run this gauntlet yourself, in one sitting, checking exit codes and output sanity after each:

The gauntlet — every line must exit without a traceback

```
# the listed ones (must all be clean - Volume 2 table)
uv run python -m src search "" --k 10
uv run python -m src search "xqzt vprw qq" --k 10
uv run python -m src search "scheduler" --k 0
uv run python -m src search_dataset --dataset_path missing.json
echo '{not json}' > /tmp/bad.json
uv run python -m src search_dataset --dataset_path /tmp/bad.json

# the inventive ones
uv run python -m src search "scheduler" --k -5
uv run python -m src search "scheduler" --k 100000
uv run python -m src search "$(python3 -c 'print("a"*100000)')'"
uv run python -m src search 'DROP TABLE; -- ; rm -rf /'
uv run python -m src search '???' --k 3
uv run python -m src index --max_chunk_size 1
uv run python -m src index --max_chunk_size -7
uv run python -m src answer "" --k 5
uv run python -m src evaluate --student_search_results_path \
  data/datasets/AnsweredQuestions/dataset_docs_public.json \
  --dataset_path data/datasets/AnsweredQuestions/dataset_docs_public.json
```

- **k larger than the corpus:** `heapq.nlargest` just returns fewer — verify the output's `k` field still echoes the request and the moulinette accepts shorter lists (it does; recall just caps).
- **The 100k-character query:** `analyze()` is linear and regex-based; this should cost milliseconds, not hang. If it hangs, you have catastrophic backtracking in a custom regex — the patterns in Volume 2 are safe.
- **Shell-metacharacter queries:** they arrive as ordinary strings (Fire quotes them); the only risk is if you ever pass query text to a shell — never do.
- **max_chunk_size 1:** the clamp keeps it legal; indexing becomes slow and silly but must terminate (the hard-split step guard). If the reviewer tries it, a progress bar that visibly advances is your friend.
- **evaluate with the wrong file as results:** a ground-truth dataset is not `StudentSearchResults`; `pydantic` rejects it and the boundary prints a clean format error — the correct behaviour.

The mutation drill

The recode step (subject ch. X.1) asks for a small live modification. Practice the plausible ones now, each a <10-minute change on the Volume 2 codebase, and each touching a different seam:

1. Add a `--json` flag to search that prints raw JSON instead of Fire's rendering (touch: `cli.py` only).
2. Change the default `k` to 5 across the CLI (`cli.py` signatures).
3. Add a new stopword and show which index artifacts must be rebuilt and why (analyzer + re-index reasoning).
4. Return the BM25 score alongside each source (extend an internal model — the subject allows extra fields; know that sentence).
5. Exclude a directory (e.g. `tests/`) from indexing via a CLI option (discovery filter + plumb the option through).
6. Make evaluate print per-question recall, worst five first (`evaluation.py` loop + sort).

Each drill rehearses the same defense skill: locating the one seam where a change belongs, in code you wrote, under mild time pressure. If any drill sends you searching through your own files for more than a minute, that seam is where your mental model is thin — re-read the matching Volume 2 chapter before the real thing.

CHAPTER 6

Bonus 1: Semantic Embeddings

A vector index beside the lexical one — vocabulary mismatch meets its counterweight.

Rules of engagement for all five bonuses, from the subject: they are graded only after the mandatory part validates completely; each must run on a CPU-only campus machine; each must work, not merely be described; and flake8/mypy discipline extends to bonus files. Order them by value per effort: this chapter and the next (semantic + hybrid) move your actual recall; caching and incremental indexing are clean engineering wins; the HTTP API is a pleasant demo. Do not start any of them before Volume 2's checklist is fully green.

Design

Add a second index: a matrix of L2-normalized MiniLM embeddings, one row per existing chunk, built right after the BM25 index and persisted beside it. Retrieval gains a mode: embed the query, dot-product against the matrix, take top-k (Volume 1 ch. 10 covered every idea here — bi-encoders, cosine as dot product, why brute force suffices at 30k chunks). The dependency is sentence-transformers; all-MiniLM-L6-v2 is ~90 MB and CPU-friendly, exactly the model class the subject names.

src/semantic.py — complete bonus 1

```
"""Semantic index (src/semantic.py) - bonus 1."""
from pathlib import Path
from typing import List, Optional, Tuple

import numpy as np

from src.indexer import PROCESSED_DIR, Bm25Index
from src.models import MinimalSource

EMB_FILE = PROCESSED_DIR / "embeddings.npy"
EMB_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
_ENCODER = None # loaded once per process

def _encoder(): # type: ignore[no-untyped-def]
    """Load the sentence-transformer once."""
    global _ENCODER
    if _ENCODER is None:
        from sentence_transformers import SentenceTransformer
        _ENCODER = SentenceTransformer(EMB_MODEL, device="cpu")
    return _ENCODER

def build_semantic_index(index: Bm25Index) -> None:
    """Embed every chunk and persist the matrix (float32)."""
    texts = [c.text for c in index.chunks]
    matrix = _encoder().encode(
        texts,
```

```

        batch_size=64,
        show_progress_bar=True,      # tqdm inside
        normalize_embeddings=True,    # cosine == dot
    ).astype(np.float32)
    np.save(EMB_FILE, matrix)

def semantic_search(
    index: Bm25Index, query: str, k: int = 10
) -> List[Tuple[int, float]]:
    """(chunk_id, cosine) for the top-k semantic neighbours."""
    if k <= 0 or not query.strip() or not EMB_FILE.exists():
        return []
    matrix = np.load(EMB_FILE, mmap_mode="r")
    q = _encoder().encode([query], normalize_embeddings=True)[0]
    scores = matrix @ q.astype(np.float32)
    top = np.argpartition(scores, -min(k, len(scores)))[-k:]
    order = top[np.argsort(scores[top])[:-1]]
    return [(int(i), float(scores[i])) for i in order]

```

Integration and cost accounting

- Wire-up: index --semantic flag calls build_semantic_index after save_index; search --mode semantic maps chunk ids to MinimalSources exactly as the lexical path does — the output contract never changes.
- Index-time cost: one forward pass per chunk. On CPU expect roughly 5-15 minutes for ~30k chunks — which is why the semantic build is opt-in behind a flag and NOT part of the mandatory index command: the 5-minute budget applies to the graded path. Say this at the defense before being asked.
- Query-time cost: one 384-dim encode (~10-30 ms CPU) plus a matrix-vector product (~10 ms). The 90-second budget laughs at it.
- mmap_mode='r' keeps the matrix on disk with OS-paged reads — cold start stays fast and memory stays flat (also the hook for bonus 4).

Note — Measure semantic-only recall with the same harness. Typical shape on this corpus: docs improves or holds, code often DROPS versus BM25 (identifiers embed poorly — Volume 1 ch. 10's failure-mode table predicted exactly this). That measured asymmetry is not a failure: it is the argument FOR hybrid, i.e. for the next chapter, and reciting it earns you both bonus points' worth of credibility.

CHAPTER 7

Bonus 2: Hybrid Retrieval

Two rankers with opposite blind spots, fused with twelve lines of Reciprocal Rank Fusion.

Volume 1 ch. 10 derived RRF and why rank fusion beats score mixing (no calibration between BM25's unbounded scores and cosine's [-1,1]). Implementation is nearly trivial once bonuses share the chunk-id space — which they do, since the semantic matrix rows are indexed by the same ids as `index.chunks`:

src/hybrid.py — complete bonus 2

```
"""Hybrid fusion (src/hybrid.py) - bonus 2."""
from typing import Dict, List, Tuple

from src.indexer import Bm25Index
from src.models import MinimalSource
from src.retriever import bm25_scores
from src.semantic import semantic_search

RRF_K = 60
CANDIDATES = 50 # depth taken from each ranker before fusing

def hybrid_search(
    index: Bm25Index, query: str, k: int = 10
) -> List[MinimalSource]:
    """Fuse lexical and semantic rankings with RRF."""
    if k <= 0:
        return []
    lex = sorted(bm25_scores(index, query),
                  key=lambda p: p[1], reverse=True)[:CANDIDATES]
    sem = semantic_search(index, query, CANDIDATES)
    fused: Dict[int, float] = {}
    for ranking in (lex, sem):
        for rank, (chunk_id, _score) in enumerate(ranking, start=1):
            fused[chunk_id] = (
                fused.get(chunk_id, 0.0) + 1.0 / (RRF_K + rank)
            )
    best = sorted(fused.items(), key=lambda p: p[1], reverse=True)
    return [
        MinimalSource(
            file_path=index.chunks[cid].file_path,
            first_character_index=index.chunks[cid].start,
            last_character_index=index.chunks[cid].end,
        )
        for cid, _ in best[:k]
    ]
```

- CANDIDATES=50 is the fusion depth: deep enough that a document one ranker loves and the other merely likes still fuses well; shallow enough to stay O(small). Sweep {20, 50, 100} with the harness if curious — the curve is flat.

- A chunk absent from one ranking simply gets no contribution from it — RRF handles asymmetric coverage with zero special-casing (semantic may be disabled entirely: hybrid then degrades to BM25 order, gracefully).
- Expose as search --mode hybrid and, if it measures best, make it the default for search_dataset — the graded path may use any retrieval method as long as one of TF-IDF/BM25 is implemented, which yours is; hybrid CONTAINS it. Quote that reading of the subject at the defense with the measured table in hand.

Note — Expected result on this corpus: hybrid meets or beats the better of its two parents on both datasets — lexical rescues identifier queries that embeddings blur; semantic rescues paraphrases that strings miss. If your hybrid UNDERPERFORMS BM25 on code, your semantic ranking is polluting the fusion: raise RRF_K (damping) or fuse only when the semantic top score clears a threshold.

CHAPTER 8

Bonus 3: Incremental Indexing

Re-index only what changed. Hash files, diff manifests, surgically replace postings.

Full re-indexing costs minutes; editing one doc file should cost milliseconds. The clean design keeps a manifest of per-file content hashes inside the persisted index, and on an incremental run: hash all current files, diff against the manifest, drop the chunks of changed/deleted files, chunk-and-add the changed/new ones, and recompute the derived statistics (avgdl). The subtlety is that Volume 2's postings map values are keyed by dense chunk ids — deletion must not renumber ids that other structures reference, so we tombstone instead of compacting:

src/incremental.py — complete bonus 3

```

"""Incremental indexing (src/incremental.py) - bonus 3."""
import hashlib
from pathlib import Path
from typing import Dict, Set

from tqdm import tqdm

from src.chunkers import chunk_markdown, chunk_python
from src.indexer import (
    Bm25Index, RAW_ROOT, _add_chunk, iter_corpus_files,
    load_index, read_text, save_index,
)

def _digest(text: str) -> str:
    """Stable content hash of a file's decoded text."""
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

def incremental_update(max_chunk_size: int = 2000) -> str:
    """Re-index only files whose content changed."""
    index = load_index()
    manifest: Dict[str, str] = getattr(index, "manifest", {})
    seen: Set[str] = set()
    changed = 0
    for path, kind in tqdm(list(iter_corpus_files(RAW_ROOT)),
                           desc="Scanning", unit="file"):
        rel = path.as_posix()
        text = read_text(path)
        digest = _digest(text)
        seen.add(rel)
        if manifest.get(rel) == digest:
            continue # unchanged - the common case
        _remove_file(index, rel)
        chunker = chunk_python if kind == "python" else chunk_markdown
        for chunk in chunker(rel, text, max_chunk_size):
            _add_chunk(index, chunk)
        manifest[rel] = digest
        changed += 1
    for gone in set(manifest) - seen: # deleted files
        _remove_file(index, gone)

```

```

        del manifest[gone]
        changed += 1
    index.manifest = manifest # type: ignore[attr-defined]
    live = [n for n in index.doc_len if n >= 0]
    index.avgdl = sum(live) / len(live) if live else 0.0
    save_index(index)
    return f"Incremental update: {changed} file(s) re-indexed"

def _remove_file(index: Bm25Index, rel: str) -> None:
    """Tombstone every chunk belonging to one file."""
    doomed = {
        cid for cid, c in enumerate(index.chunks)
        if c.file_path == rel and index.doc_len[cid] >= 0
    }
    if not doomed:
        return
    for postings in index.postings.values():
        for cid in doomed & postings.keys():
            del postings[cid]
    for cid in doomed:
        index.doc_len[cid] = -1 # tombstone: never scored again

```

- Tombstones (doc_len = -1) keep ids stable so the semantic matrix and any caches stay aligned; scoring never sees them because their postings entries are gone. Add a compaction path (full rebuild) for when tombstones exceed, say, 20% — and mention the threshold, it shows systems taste.
- Content hashes, not mtimes: a git checkout touches every mtime without changing bytes; sha256 of the decoded text is immune and costs microseconds per file.
- The demo that earns the point: time index (minutes), touch one .md file with a real edit, time index_update (seconds), then search a term you just added and watch it surface. Script the demo so defense nerves cannot fumble it.
- Storing the manifest on the index object keeps one artifact on disk; if you prefer a clean dataclass field over setattr, add manifest: Dict[str, str] = field(default_factory=dict) to Bm25Index — one line, and mypy prefers it too.

CHAPTER 9

Bonus 4: Caching

Two caches, two different problems: cold start and repeated queries.

The subject names both targets: “cache the index and query results to speed up cold start and repeated queries.” They are different engineering problems. The index cache is about process lifetime — load once, reuse across commands within a process (already done in batch mode) and make the on-disk load itself cheap. The query cache is about repeated work — same query, same k, same index must not re-score.

src/cache.py — complete bonus 4

```

"""Caching (src/cache.py) - bonus 4."""
import functools
import json
import time
from pathlib import Path
from typing import List, Optional, Tuple

from src.indexer import INDEX_FILE, Bm25Index, load_index
from src.models import MinimalSource
from src.retriever import search

CACHE_DIR = Path("data/processed/cache")
_INDEX: Optional[Tuple[float, Bm25Index]] = None

def cached_index() -> Bm25Index:
    """Process-level index cache, invalidated by file mtime."""
    global _INDEX
    stamp = INDEX_FILE.stat().st_mtime
    if _INDEX is None or _INDEX[0] != stamp:
        _INDEX = (stamp, load_index())
    return _INDEX[1]

def cached_search(query: str, k: int) -> List[MinimalSource]:
    """Disk-backed query cache keyed by (query, k, index stamp)."""
    index = cached_index()
    stamp = int(INDEX_FILE.stat().st_mtime)
    key = f"{stamp}-{k}-{hash(query) & 0xffffffff:08x}.json"
    path = CACHE_DIR / key
    if path.exists():
        with path.open("r", encoding="utf-8") as fh:
            return [MinimalSource(**s) for s in json.load(fh)]
    sources = search(index, query, k)
    CACHE_DIR.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as fh:
        json.dump([s.model_dump() for s in sources], fh)
    return sources

```

- **Invalidation is the whole design:** both caches key on the index file's mtime, so rebuilding the index (or an incremental update — bonus synergy) silently invalidates everything stale. When

asked the classic 'what are the two hard problems' question, you now have a concrete invalidation story instead of a joke.

- The disk cache makes repeated single-query CLI calls fast across PROCESSES — that is the visible demo: time src search 'lora' twice, second call returns in interpreter-startup time.
- hash(query) is salted per process in Python — for a purely local cache that only means occasional misses across runs; use `hashlib.sha1(query.encode()).hexdigest()[:16]` instead if you want stable keys, and say why. (This is a deliberate discussion point: noticing it IS the bonus.)
- In-process LRU for batch mode costs one decorator on a helper (`functools.lru_cache(maxsize=512)` over (query, k) tuples) — datasets with duplicate questions benefit; measure and report.

CHAPTER 10

Bonus 5: A Local HTTP API

The same engine, one thin FastAPI layer — so anything, not just the CLI, can drive it.

The subject asks for a small local HTTP API exposing querying and answering. The architecture rule that makes this a one-evening bonus: the API imports the SAME functions the CLI uses — zero logic is duplicated, the layer only translates HTTP to calls:

src/api.py — complete bonus 5

```
"""Local HTTP API (src/api.py) - bonus 5.

Run: uv run uvicorn src.api:app --port 8000
"""
from typing import List

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

from src.cache import cached_index, cached_search
from src.models import MinimalSource

app = FastAPI(title="RAG against the machine")

class SearchResponse(BaseModel):
    """Wire format for /search."""
    query: str
    k: int
    sources: List[MinimalSource]

class AnswerResponse(SearchResponse):
    """Wire format for /answer."""
    answer: str

@app.get("/health")
def health() -> dict[str, str]:
    """Liveness probe; also warms the index cache."""
    cached_index()
    return {"status": "ok"}

@app.get("/search", response_model=SearchResponse)
def http_search(q: str = "", k: int = 10) -> SearchResponse:
    """Top-k sources for a query."""
    if k < 0:
        raise HTTPException(400, "k must be >= 0")
    return SearchResponse(
        query=q, k=k, sources=cached_search(q, k)
    )

@app.get("/answer", response_model=AnswerResponse)
```

```
def http_answer(q: str = "", k: int = 5) -> AnswerResponse:
    """Grounded answer for a query."""
    from src import generator # heavy import stays lazy
    sources = cached_search(q, k)
    answer = generator.answer_question(cached_index(), q, sources)
    return AnswerResponse(query=q, k=k, sources=sources, answer=answer)
```

- pydantic response models mean the API's wire format is validated by the same machinery as your files — and FastAPI generates interactive docs at /docs, which is the flashiest thirty seconds of any defense demo: curl or the browser, live, against your real index.
- Error semantics mirror the CLI's: degenerate q returns an empty source list with 200; a negative k is a 400 with a clean message; a missing index surfaces as a 500 whose detail names the fix — translate FileNotFoundError in an exception handler if you want it pretty (three lines).
- fastapi and uvicorn go into a [project.optional-dependencies] extra (api = [...]), keeping the mandatory uv sync lean. Document the extra in the README's bonus section.
- Demo script: health, search 'How do I enable LoRA?', answer the same — three curls saved in a file, rehearsed once.

Note — Bonus scoring reality check, one line each, as an evaluator will ask: 1 - where is the vector index and what model built it? 2 - write the RRF formula. 3 - what invalidates your manifest entry? 4 - what keys your caches and what invalidates them? 5 - which endpoint reuses which CLI function? If any answer takes you more than one breath, re-read that chapter's code — you wrote it, own it.

CHAPTER 11

A Complete Tuning Case Study

From 0.58 to 0.86 docs recall@5 in six measured iterations — a realistic composite of how this optimization actually goes.

This chapter narrates a full tuning campaign on the docs dataset, from a naive baseline to a comfortable pass. The numbers are a realistic composite (your corpus statistics will differ in detail, not in shape), and every move follows the triage discipline of chapter 2: classify, fix the biggest class, re-measure. Read it as a template for your own lab notebook.

Iteration 0 — the naive baseline

Configuration: fixed 1000-character windows for ALL file types (no structure awareness), analyzer = lowercase word-split only (no identifier splitting, no stoplist), BM25 defaults $k1=1.5$, $b=0.75$.

Baseline: failing, with a diagnosis attached

```
recall@1 0.31  recall@3 0.47  recall@5 0.58  recall@10 0.66
triage over 100 questions, 138 truth sources:
  FILE-MISS: 31  SPAN-MISS: 17  RANK-MISS: 10  found: 80
```

Reading: the staircase keeps climbing to @10 (+0.08), so ordering is mediocre too, but the dominant class is FILE-MISS — retrieval never touches the right file for 31 sources. Inspecting ten of them shows two sub-causes: questions using spaced words for compound tokens (“graceful shutdown timeout” vs graceful_shutdown_timeout in a config table), and matches diluted across the enormous unfiltered index (binary-ish files and vendored assets were being indexed as text).

Iteration 1 — fix discovery filters

Change: proper extension whitelist and the 2 MB size cap (Volume 2 ch. 3). Index shrinks 34%; avgdl stabilizes; IDF sharpens because junk chunks no longer inflate document frequencies of common terms.

Cleaning the corpus is worth five points before touching any algorithm

```
recall@5: 0.58 -> 0.63  FILE-MISS 31 -> 26  RANK-MISS 10 -> 7
```

Iteration 2 — identifier splitting in the analyzer

Change: split snake_case/camelCase, emit both forms (Volume 2 ch. 4). Re-index (analyzer changes always mean re-index — the index stores analyzed terms).

The single biggest lever in the whole campaign, exactly as theory predicted

```
recall@5: 0.63 -> 0.71  FILE-MISS 26 -> 15
docs gains are real but the CODE dataset jumps even harder:
code recall@5: 0.34 -> 0.49
```

Iteration 3 — structural chunking

Change: heading-based Markdown chunker and AST Python chunker replace fixed windows (Volume 2 ch. 5-6), target size raised toward the ceiling (~1800).

Span misses collapse when boundaries follow structure

```
recall@5: 0.71 -> 0.78      SPAN-MISS 17 -> 6
recall@10 - recall@5 gap: 0.07 -> 0.04
```

Reading: SPAN-MISS was chunk boundaries cutting through answer regions; sections-as-chunks fixed most. The narrowing @5/@10 gap says ordering also improved — coherent chunks score more sharply (higher within-chunk term density for the on-topic terms).

Iteration 4 — path-term boosting

Change: append analyzed file-path terms to each chunk's term list at indexing (chapter 3, knob 5).
Cost: a dozen lines, re-index.

Over the threshold — but 0.82 is 'passing', not 'comfortable'

```
recall@5: 0.78 -> 0.82      FILE-MISS 15 -> 11
```

Iteration 5 — k1/b grid

Change: sixteen-point grid, no re-indexing. Best cell for docs: k1=1.2, b=0.6 (mildly less TF reward, mildly less length punishment than defaults — plausible for section-sized prose chunks).

Parameter tuning is real but small: two points, exactly as advertised

```
recall@5: 0.82 -> 0.84      RANK-MISS 7 -> 4
```

Iteration 6 — per-file dedup in top-k

Change: triage shows several questions whose top-5 contains three near-identical chunks from one README (overlap artifacts), crowding out a second ground-truth file. Fix at retrieval: cap results per file at 2 within the top-k (take best two per file, fill remaining slots by score).

Comfortable. Remaining misses are genuine vocabulary mismatch — bonus 2's job

```
recall@5: 0.84 -> 0.86      multi-source questions improve most
final staircase: 0.52 / 0.74 / 0.86 / 0.90
final triage:      FILE-MISS 9  SPAN-MISS 3  RANK-MISS 2
```

What the campaign teaches

1. Gains came in the order theory ranks the levers: corpus hygiene, analyzer, chunker, boosts, parameters, top-k shaping. Five of six iterations were INDEXING-side changes; only one touched scoring parameters. Students who start at the k1/b grid spend a day earning two points while twenty sit in the analyzer.
2. Every iteration was one change, one re-measure, one triage. When iteration 4 also nudged code recall down half a point (path terms mildly diluted code chunks), the notebook caught it and

the decision — keep, net positive — was conscious.

3. The residual FILE-MISSes at the end are the irreducible lexical core: questions that paraphrase beyond term overlap. That is the measured, principled motivation for the semantic and hybrid bonuses, and presenting it this way at the defense turns your bonus section into the conclusion of an experiment rather than a checkbox.

Note — Reproduce this chapter's table format in your README's performance analysis. An evaluator who sees iteration-by-iteration numbers with triage counts stops probing your understanding of retrieval — the table already answered.

CHAPTER 12

Question Post-Mortems

Eight failure archetypes, dissected one real-shaped question at a time. Learn to see which archetype is looking at you.

Triage gives you classes; this chapter gives you the archetypes inside them. Each post-mortem below is one question (representative of the public datasets' style), the failure as observed, the root-cause chain, and the fix family. These are the eight shapes that cover essentially every miss you will meet.

1 — The paraphrase (FILE-MISS, docs)

Q: “How can I make vLLM start faster after the first run?” **Truth:** docs section on prefix caching / compilation cache. **Observed:** top-k full of benchmark files mentioning “faster”. **Chain:** the answer section says “cold start”, “warmup”, “compilation” — zero content words shared with the question. Pure vocabulary mismatch; no lexical knob fixes it honestly. **Fix family:** semantic/hybrid retrieval (the embedding puts “start faster after first run” near “cold start warmup”); secondarily, stemming does nothing here — know when a lever is the wrong lever.

2 — The verbatim identifier (should be easy; missed by over-normalization)

Q: “What does max_num_batched_tokens control?” **Truth:** the config definition plus one docs table row. **Observed:** hits everywhere the PARTS appear — “max”, “num”, “batched”, “tokens” are common — the definition ranks 8th. **Chain:** the analyzer split the identifier but did NOT keep the intact token (or stemmed it), so the question's verbatim mention matched only through common fragments with low IDF. **Fix family:** keep-both-forms in the analyzer — the intact token's df is tiny, its IDF huge, and the definition rockets to rank 1. This archetype is the mirror image of #1: together they ARE the subject's hint about paraphrase vs verbatim.

3 — The split table (SPAN-MISS, docs)

Q: “Which quantization methods does vLLM support?” **Truth:** a long Markdown table under docs/features/. **Observed:** right file retrieved, IoU 0.03 — a whisker under the bar. **Chain:** the table exceeded max size; the hard splitter cut it mid-table WITHOUT overlap, and the truth span straddles the cut. **Fix family:** overlap on hard splits (Volume 2 ships 200 chars — verify yours actually applies on this path); or larger chunks so the table survives whole.

4 — The scattered class (SPAN-MISS/RANK-MISS, code)

Q: “How does the scheduler decide to preempt a sequence?” **Truth:** one method deep inside a 3000-line class. **Observed:** fixed-window era: four windows of the class in top-20, none overlapping the method enough. AST era: the method chunk ranks 2. **Chain:** meaning lived in a syntactic unit; windows sliced it agnostically. **Fix family:** AST chunking with per-method descent — this archetype is why the subject mandates a code-specific strategy.

5 — The crowd-out (RANK-MISS, multi-source)

Q: two truth sources: a docs page AND the implementing module. **Observed:** docs source found at rank 1; ranks 2-5 are four more chunks of the SAME docs page; the code source sits at rank 7. $\text{recall@5} = 0.5$. **Chain:** one strong file monopolizes top-k. **Fix family:** per-file cap in top-k (case study iteration 6); the fused ranking of bonus 2 also naturally diversifies.

6 — The stopworded signal (FILE-MISS, code)

Q: “What happens WHILE a request is being preempted?” — with an aggressive 150-word stoplist that dropped “while”, “during”, “being”. **Chain:** fine — those are noise. But the same list also dropped “not”, “no”, “off”, and a question about “turning OFF prefix caching” lost its only discriminating term against pages about turning it ON. **Fix family:** short stoplist (~30 words), and never stopword negations. Measured in chapter 3, knob 4.

7 — The wrong-prefix ghost (systemic zero)

Q: all of them. **Observed:** valid: True, every recall 0.000 — the most alarming and most fixable screen in the project. **Chain:** paths serialized as `vllm-0.10.1/docs/...` (missing `data/raw/`) or absolute paths from a resolved Path. Verbatim comparison fails universally. **Fix family:** Volume 2 ch. 3’s relative-from-discovery rule plus the pre-flight script from chapter 1 here. Archetype included because someone reading this will still hit it — make it a two-minute fix, not a lost evening.

8 — The question your corpus genuinely cannot answer

Q: occasionally a question’s truth source lives in a file type you chose not to index (a `.inc`, an unusual extension, a just-over-cap file). **Observed:** permanent FILE-MISS immune to all tuning. **Chain:** coverage hole. **Fix family:** run the coverage check — every ground-truth `file_path` in the public datasets must be discoverable by your walker:

The coverage check — run it the day your walker is written, and after every filter change

```
from src.indexer import iter_corpus_files
from src.models import AnsweredQuestion, RagDataset, load_model

indexed = {p.as_posix() for p, _ in iter_corpus_files()}
for name in ("docs", "code"):
    ds = load_model(Path(f"data/datasets/AnsweredQuestions/"
                        f"dataset_{name}_public.json"), RagDataset)
    truth_files = {s.file_path for q in ds.rag_questions
                   if isinstance(q, AnsweredQuestion)
                   for s in q.sources}
    missing = truth_files - indexed
    print(name, "uncovered truth files:", sorted(missing) or "none")
```

If this prints anything but “none”, no amount of scoring brilliance recovers those questions. It is the first sanity gate of every tuning campaign — the case study's iteration 1 was this archetype wearing a different hat.

CHAPTER 13

Beyond the Subject: The Next Three Techniques

Not required, occasionally decisive, always asked about by curious evaluators: re-ranking, query expansion, and structured filtering.

Nothing in this chapter is needed to validate. All three techniques are, however, (a) the standard next steps in real retrieval systems, (b) cheap enough to prototype with your existing harness if you have a spare evening, and (c) excellent material for the 'design decisions' README section even as roads-not-taken — provided you can say WHY you didn't take them.

Re-ranking with a cross-encoder

Volume 1 ch. 10's note introduced the pattern: a bi-encoder or BM25 retrieves a candidate set (say 50), and a **cross-encoder** — a transformer reading query and chunk TOGETHER — re-scores those 50 with far higher fidelity, at ~20-50 ms per pair on CPU. For 200 questions x 50 candidates that is 3-8 minutes: over the 90-second budget if applied to search_dataset, which is exactly the trade-off to articulate. Where it shines here: as an answer-stage filter — re-rank only the top 10 before choosing the 3-5 snippets the generator sees. Recall (the grade) is untouched; answer grounding (the rubric) improves. Model: cross-encoder/ms-marco-MiniLM-L-6-v2, same dependency family as bonus 1.

Query expansion

Attack vocabulary mismatch from the query side: add related terms before scoring. Three escalating variants: (1) hand-curated synonym table for the domain's recurring aliases (GPU memory <-> gpu_memory_utilization, LoRA <-> adapter — ten entries cover a surprising share of docs questions); (2) pseudo-relevance feedback (RM3): run the query, take top terms of the top-3 chunks by TF-IDF-within-results, append, re-run — classic, free, occasionally unstable on short queries; (3) LLM expansion: ask Qwen to rewrite the question with technical synonyms (HyDE is the fancy cousin: generate a hypothetical answer and search for THAT). Variant 3 costs a generation per question — budget mathematics again. If you prototype only one, take (1): deterministic, explainable, and its failure mode is a no-op.

Structured filtering and routing

Your corpus has structure the retriever ignores: file type, directory, docs-vs-code. A **router** classifies the question first ("sounds like a usage question -> weight docs/; sounds like an internals question -> weight vllm/") and applies per-route boosts or filters. The honest version of chapter 3's extension-aware scoring, generalized. The trap: a misrouted question is CATASTROPHICALLY

penalized (its true territory got down-weighted), so routers need either high confidence or soft weighting. Measured soft boosts (x1.2, not exclusion) are the defensible middle. This is also the cleanest answer to the evaluator's "how would you scale this to ten corpora?" — route first, retrieve within the route.

Technique	Cost to prototype	Where it helps here	Why it's not in the mandatory build
Cross-encoder re-rank	1 evening	answer grounding; residual RANK-MISS	90 s budget if applied at search; grade measures recall, not ordering quality
Synonym expansion	2 hours	residual FILE-MISS paraphrases	hand-curation is corpus-specific; hybrid attacks the same class more generally
RM3 feedback	1 evening	docs long-tail	instability on short queries; another knob to defend
LLM expansion / HyDE	1 evening	hardest paraphrases	generation cost per query; nondeterminism in the graded path
Routing	half a day	cross-corpus noise	misroute risk; per-dataset thresholds already separate concerns

CHAPTER 14

Defense Drill II: Thirty Questions on Practice

Volume 1 drilled the theory. This drills everything an evaluator asks about YOUR build, YOUR numbers, YOUR choices.

Format as before: cover the right column, answer aloud in under thirty seconds, re-read the referenced chapter for anything that stumbles. Questions marked [Y] expect YOUR numbers — placeholders here, your lab notebook's values out loud.

Build and pipeline

Question	Compressed answer
Walk me through 'python -m src index'.	Fire dispatch -> clamp size -> discovery walk (filters) -> per-kind chunker -> analyzer -> postings fold -> avgdl -> pickle + meta. (Vol 2 ch. 14 verbatim.)
Where exactly is the 2000-char limit enforced?	By construction in both chunkers' packing, asserted in <code>_make</code> , clamped at the CLI, and pre-flight-checked before the moulinette. Four layers, one invariant.
Why is the analyzer a single shared function?	Index terms and query terms must be produced identically or postings never match; one import point makes asymmetry impossible and one unit test locks it.
Why pydantic only for exchanged models, dataclass for Chunk?	Subject exempts service internals; Chunk is a hot-path type built tens of thousands of times; validation belongs at I/O boundaries, which all pass through <code>load/save_model</code> .
What happens on a file your AST parser can't parse?	<code>SyntaxError</code> is caught; the text chunker takes over; the file stays indexed. Skipping would create an unreachable ground-truth source.
Why store chunk text in the index?	The generator needs it; re-reading files would re-open encoding/offset consistency risks; tens of MB is cheap.
[Y] Your index: how many chunks, how many terms, how long to build?	~N chunks / ~T terms / ~M minutes - from <code>index_meta</code> .
What would you change for a 100x larger corpus?	Postings to disk (SQLite/LMDB or sharded), docs streamed not held, ANN index for dense, and routing (ch. 13). The current all-in-RAM design is right-sized for 30k chunks, wrong at 3M.

Scoring and tuning

Question	Compressed answer
[Y] Your recall staircase on both public datasets?	docs a/b/c/d, code w/x/y/z - and the shape's diagnosis.
Which change moved recall most, and by how much?	Identifier splitting (+8 docs, +15 code in the case-study composite; YOUR numbers here). Then structural chunking.
Why does a big @10-@5 gap point at scoring, not chunking?	The candidates exist in top-10, so indexing found them; only their ORDER keeps them out of top-5 - that's k1/b, boosts, dedup.
Defend your k1 and b.	Grid-searched {0.9..2.0}x{0.3..0.9} with the harness; best cell k1=?, b=?; movement was ~2 points; defaults were the prior, measurement the posterior.
Why cap results per file in top-k?	Multi-source questions: five chunks of one file can match at most one truth source; diversity converts wasted slots into recall on the second file.
Your residual misses - why do they persist?	Triage shows FILE-MISS paraphrases beyond lexical reach; that's the measured motivation for semantic/hybrid, whose numbers are in the README.
Could you overfit the public datasets? How do you know you didn't?	Yes - question-specific hacks. Every change here is a general principle (splitting, structure, boosts) argued from theory first, measured second; nothing keys on question strings or specific paths.

Generation and bonuses

Question	Compressed answer
Why feed 5 chunks to Qwen when you retrieved 10?	JSON reports retrieval; prompt is editorial. Small models degrade with marginal context; 3-5 clean snippets answer best. Both facts from Vol 1 ch. 12, verified by eyeballing answers.
How do you keep thinking-mode text out of answers?	enable_thinking=False at the template, plus a defensive strip of anything before </think> in _clean.
Your semantic index: model, dimension, build cost, why not in the graded index command?	all-MiniLM-L6-v2, 384-d, minutes on CPU - opt-in flag because the 5-minute budget binds the graded path.
Write RRF and say why rank, not score, fusion.	sum of 1/(60+rank) per ranking; BM25 and cosine scales are incomparable, ranks always are; agreement dominates.
What invalidates each cache?	Index file mtime keys both the in-process index cache and the on-disk query cache; rebuild or incremental update changes mtime, everything stale dies.
Incremental indexing: why hashes, why tombstones?	mtimes lie after checkouts; sha256 of content doesn't. Tombstones keep chunk ids stable so the embedding matrix and caches stay aligned; compaction at ~20%.

Note — Final rehearsal protocol, night before: fresh clone, full walkthrough, gauntlet, one mutation drill drawn at random, then thirty minutes of these tables aloud with your partner. Sleep. You are more ready than the median evaluator has ever seen.

CHAPTER 15

The README, Section by Section

Seven required sections beyond the basics. Write it for a recruiter in a hurry and an evaluator with a checklist — the same document serves both.

The subject specifies the README more precisely than most students notice, and evaluators grade it with the list open. The complete required inventory, in a sensible order:

Section	Requirement source	What 'good' looks like
First line	must be italicized, exact wording: 'This project has been created as part of the 42 curriculum by <login1>...'	one line, italics, real logins
Description	base requirement	3-6 sentences: what RAG is, what this system does, the corpus, the model, the thresholds met
Instructions	base requirement	uv sync; make rules; the four-command walkthrough copy-pasteable
System architecture	project-specific	the four stages, your modules mapped to them, one diagram or ASCII flow
Chunking strategy	project-specific	both strategies, boundary rules, the 2000 invariant, chosen sizes WITH numbers
Retrieval method	project-specific	BM25 formula, your k1/b, analyzer behaviour, any boosts
Performance analysis	project-specific	recall table from your sweeps; index/batch timings; the staircase discussed
Design decisions	project-specific	5-8 decisions, each with the rejected alternative and why
Challenges faced	project-specific	real failures from your lab notebook and their fixes
Example usage	project-specific	single query, dataset run, evaluate — with real output snippets
Resources	base requirement	classic references (BM25 paper, IR book, RAG paper...) AND the mandated description of how AI was used, task by task

A skeleton that satisfies the list

README.md skeleton — fill every table with YOUR measured numbers

This project has been created as part of the 42 curriculum by login1, login2.

RAG against the machine

Description

Retrieval-Augmented Generation over the vLLM 0.10.1 codebase...

Instructions

```bash

uv sync

make lint && make run

uv run python -m src index --max\_chunk\_size 2000

...

```

System architecture

question -> [analyzer] -> [BM25 index] -> top-k sources

-> [context builder] -> [Qwen3-0.6B] -> grounded answer

...

Chunking strategy

Retrieval method

Performance analysis

max_chunk_size	docs R@5	code R@5	index time
800	0.xx	0.xx	x m

...

Design decisions

Challenges faced

Example usage

Bonus features

Resources & AI usage

The AI-usage paragraph

The subject requires “a description of how AI was used — specifying for which tasks and which parts of the project”. Write it plainly and specifically: which tools, for what (boilerplate, debugging discussion, doc drafting), what you verified yourself, and what you deliberately wrote by hand. Under the subject's AI-instructions chapter, unexplainable AI-generated code is a failing offense while honest, understood use is encouraged — the paragraph is your chance to demonstrate you are on the right side of that line. Vagueness (“AI helped with some parts”) reads as concealment; specificity reads as mastery.

Watch out — The README must be in English, at the repository root, and the first line's wording is checked pedantically — italics, the exact phrase, your real logins. Zero-cost points; do not donate them.

CHAPTER 16

The Defense

Forty-five minutes to prove three things: it works, it scores, and you understand every line of it.

How the session typically flows

1. **Clone and setup (5 min).** Fresh clone, uv sync, make lint. Anything red here poisons the room. You rehearsed this in Volume 2 ch. 18.
2. **The automated run (10 min).** The reference exam scripts drive index -> search_dataset -> moulinette on datasets THEY choose, with THEIR paths. This is why nothing is hard-coded and the layout is exact. The thresholds decide the mandatory part here.
3. **Exploration (10 min).** The evaluator pokes: edge cases from the gauntlet (ch. 5), odd queries, a look at data/processed/, questions about output files.
4. **Theory and code reading (10-15 min).** Explain BM25, walk a command's flow (Volume 2 ch. 14 is this rehearsal), justify chunk sizes with your measurements, answer 'why X and not Y' for the big five decisions (Volume 1 ch. 16).
5. **Recode (if invoked) (5-10 min).** One of the mutation drills from ch. 5, or a neighbour of one.
6. **Bonuses (only if mandatory is flawless).** Demo semantic/hybrid with measured recall, incremental with the timed script, caching with the double-timed query, API with the three curls.

Answers you should have loaded, verbatim-ready

- **“Why is code recall harder than docs?”** Code vocabulary is compound identifiers and question vocabulary is prose — mismatch is structural; docs carry their own headings as query-shaped titles. My analyzer splits identifiers while keeping the originals, which closed most of the gap; here are the before/after numbers.
- **“Why didn't you use embeddings for the mandatory part?”** The subject mandates a lexical method; embeddings on identifier-heavy corpora measurably underperform BM25 for verbatim queries; I implemented them as bonus 1 and fused with RRF as bonus 2, and the hybrid's numbers are in the README.
- **“What happens if I ask a question with no answer in the corpus?”** Retrieval returns the least-bad top-k (recall is undefined without ground truth); generation's escape-hatch instruction plus the empty-context guard produce an honest 'not in the provided context' rather than a hallucination.

- **“Why pickle and not JSON for the index?”** One-call round-trip of the whole structure, sub-second load, internal-only artifact regenerated by one command; the wire formats the grader touches are all pydantic-validated JSON. (If they press on pickle safety: it is a local file MY code wrote; nothing untrusted is ever unpickled.)
- **“Prove the analyzer is symmetric.”** Same function object imported by indexer and retriever — here is the import in each file, and here is the unit test that locks the contract.

Conduct under fire

When something breaks live — it happens — narrate your debugging instead of freezing: “empty result on a query that should hit; let me check the analyzer output for it; ah, the term is stopworded”. Evaluators grade the recovery as highly as the absence of failure, because recovery is the transferable skill. And when you do not know an answer, say so and reason aloud from what you do know — the subject's whole AI chapter is one long warning against bluffing, and evaluators are calibrated bluff detectors.

Note — Logistics that save defenses: model weights pre-downloaded on the demo machine; the corpus and datasets in place; a terminal with the walkthrough commands in history; the README open in a tab; your lab-notebook numbers printed or memorized. Ten minutes of staging the night before buys you calm the whole session.

CHAPTER 17

The Final Checklist

Everything, one page per concern. Green means green from a FRESH CLONE, not from your working tree.

Compliance

- Python 3.10+; uv-managed; uv.lock + pyproject.toml + .python-version committed and coherent.
- Makefile: install, run, debug, clean, lint (exact flags), lint-strict; all exit 0.
- flake8 clean; mypy clean under the subject's five flags; every def annotated; PEP 257 docstrings on every function and class.
- pydantic models byte-compatible with the subject's schemas; Fire CLI; tqdm on every long loop.
- All six commands present with the subject's exact names and options; no hard-coded paths anywhere.
- Repository layout matches the evaluation tree; no data, outputs, weights, or moulinette binary committed.

Scoring

- Moulinette validates outputs (valid: True) on both public datasets.
- docs recall@5 comfortably above 0.80; code comfortably above 0.50 (margin absorbs private-set shift).
- Indexing under 5 minutes and 200-question search under 90 seconds on campus-grade hardware, measured, numbers in the README.
- Own evaluate agrees with the moulinette within rounding on both datasets.

Robustness

- The full gauntlet (ch. 5) passes: no unhandled traceback under any listed or inventive input.
- Every open() is a context manager; every error message names the fix.
- Unit tests green: chunker invariants, analyzer contract, ranking sanity, IoU edges, end-to-end smoke.

Story

- README complete per ch. 11's table, first line pedantically exact, AI-usage paragraph specific.

- You can whiteboard: the four stages, the BM25 formula with both knobs explained, the IoU matching rule, and the five decisive design choices with their measurements.
- Mutation drills rehearsed; walkthrough rehearsed from fresh clone; demo machine staged.
- Bonuses (if attempted) individually demoable in under two minutes each, with their one-breath answers ready.

CHAPTER 18

Closing: What This Project Actually Taught You

RAG was the syllabus. The lessons are bigger.

Strip away the moulinette and the thresholds and look at what you now know how to do. You met an ill-posed problem — “make a tiny model answer questions about forty thousand files” — and decomposed it into stages with crisp contracts. You learned that the glamorous component (the neural network) was the least important one to engineer, and that a regex in a tokenizer moved the outcome more than any amount of prompt poetry. You built a measurement harness before turning knobs, kept a lab notebook, and let numbers override your intuitions — which were wrong exactly as often as Volume 1 promised they would be.

These are the transferable skills: **contracts between stages** (the pydantic models were a type system for a pipeline, and every debugging session localized in one hop because of them); **evaluation-driven development** (recall@5 was this project's unit test for an entire subsystem — production ML teams live this way); **right-sizing** (a dict beat a vector database at this scale, and knowing when it stops winning is the actual expertise); and **honest engineering narrative** (the README that reports failed experiments next to successful ones is the genre you will write for the rest of your career, under names like 'design doc' and 'postmortem').

Where to go from here, if the topic caught you: build the re-ranking third stage (a cross-encoder over your hybrid top-50) and watch nDCG move; swap the corpus for your own past projects and feel how corpus personality changes every tuning decision; read the vLLM paper you indexed and realize you can now navigate its implementation with your own tool; or go the agentic route — give the model your search as a callable function and let it decide when to retrieve, which is where the field currently lives. The three volumes end here; the system is yours. Will you answer my questions? — you, now, verifiably: yes.

CHAPTER 19

Appendix: The Complete Parameter Reference

Every constant in the reference system, its effect, its safe range, and what breaks at the edges.

Every number in the Volume 2 build, collected. For each: where it lives, what it trades, the measured-sane range, and the failure mode outside it. When a reviewer asks “why 200?” about anything, the answer pattern is always: default from theory, range from measurement, edge behaviour verified.

Indexing

Parameter	Default	Trades	Safe range / edge behaviour
max_chunk_size	2000	span containment vs term-statistic sharpness	800-2000; >2000 clamped (moulinette rejects); <200 explodes chunk count and drowns top-k in fragments
hard-split overlap	200	boundary safety vs index size and near-duplicate crowding	100-300; 0 recreates archetype #3 (split table); >500 needs per-file dedup to protect top-k
MAX_FILE_BYTES	2 MB	coverage vs index bloat and build time	1-5 MB; too low risks a coverage hole - always re-run the ch. 12 coverage check after changing
extension whitelist	py/md + text set	coverage vs noise	add before removing; removal only after the coverage check proves no truth file uses it
stoplist size	~30	noise removal vs signal loss	0-40; negations never stopworded; 100+ measurably harms code questions (archetype #6)

Scoring

Parameter	Default	Trades	Safe range / edge behaviour
k1	1.5	TF reward vs saturation	0.9-2.0; 0 = binary matching (repetition ignored); >3 approaches raw-TF pathology on repetitive chunks

Parameter	Default	Trades	Safe range / edge behaviour
b	0.75	length discount strength	0.3-0.9; 0 lets near-ceiling chunks dominate smaller structural ones; 1.0 over-punishes legitimately dense sections
IDF formula	$\ln((N-df+.5)/(df+.5)+1)$	smoothing + positivity	keep the +1: without it, terms in >half the chunks score negative and REPEL matches
per-file cap in top-k	2	depth in one file vs diversity across files	1-3; 1 hurts questions whose truth spans two regions of one file; none recreates archetype #5
k (results)	10	grader looks at 5; 10 gives the answer stage headroom	always ≥ 5 for grading; the k field must echo the request

Generation

Parameter	Default	Trades	Safe range / edge behaviour
MAX_SOURCES_IN_PROMPT	5	evidence coverage vs attention dilution	3-6; 10 measurably degrades 0.6B answer focus; 1 fails multi-source questions
CONTEXT_BUDGET_CHARS	8000	evidence vs speed and focus	5000-12000; whole-snippet truncation only - never cut mid-snippet
MAX_NEW_TOKENS	256	answer completeness vs dataset run time	128-512; unlimited + thinking mode = minutes per question
temperature/top_p/top_k	0.7/0.8/20	vendor guidance (non-thinking) vs determinism	greedy+repetition_penalty is the defensible alternative; never greedy WITH thinking mode
enable_thinking	False	small reasoning gain vs 3-10x time and trace pollution	False for dataset runs; if True anywhere, _clean must strip

Bonuses

Parameter	Default	Trades	Safe range / edge behaviour
embedding model	all-MiniLM-L6-v2	quality vs CPU cost	any small sentence-transformer; bigger models blow the index-build patience budget on CPU
embedding batch_size	64	throughput vs RAM	16-128 on CPU; irrelevant to quality
RRF K	60	damping of any single ranker's top ranks	20-100; flat region - the literature default is fine, say so

Parameter	Default	Trades	Safe range / edge behaviour
fusion CANDIDATES	50	fusion depth vs cost	20-100; flat; below 20 loses cross-ranker rescues
cache key	index mtime + query + k	correctness of invalidation	content-hash the index file instead if mtime granularity ever bites (it won't here)
tombstone compaction	20%	rebuild cost vs scoring-loop waste	10-30%; pure taste - defend the existence of the threshold, not its exact value

A closing observation worth internalizing: of the twenty-odd parameters above, exactly three (`max_chunk_size`, `k1`, `b`) are the ones students argue about, and they collectively moved the case study four points. The unglamorous ones — extension whitelist, stoplist size, keep-both-forms, per-file cap — moved it twenty-four. Parameter tables look democratic; leverage never is. Know where yours concentrates, and spend your remaining hours there.

CHAPTER 20

Appendix: Operations on Campus Machines

Disk quotas, model caches, offline runs, and the ten commands that make a constrained machine behave.

The subject warns that “the deep-learning stack and the model weights can total several gigabytes”, and campus machines add their own constraints: home-directory quotas, shared hardware, sometimes flaky network. This appendix is the operations manual for running the project — and especially the defense — on such a machine.

Where the gigabytes actually go

Artifact	Location (default)	Size	Movable?
uv-managed Python + venv	.venv/ in the project	~5-7 GB with torch	yes: project location decides
torch + transformers wheels cache	~/.cache/uv (or pip)	~2-3 GB	yes: UV_CACHE_DIR
Qwen3-0.6B weights + tokenizer	~/.cache/huggingface/	~1.5 GB	yes: HF_HOME
MiniLM (bonus 1)	~/.cache/huggingface/	~90 MB	same
your BM25 index	data/processed/	~50-100 MB	already project-local
embedding matrix (bonus 1)	data/processed/	~40-60 MB	project-local
the corpus	data/raw/	~50-100 MB	provided

If the home quota is tight, point every cache at a roomier volume (a sgoinfre-style scratch space or an external disk) BEFORE first install — moving caches after the fact is re-downloading with extra steps:

Cache placement — do this once, on the machine the defense uses

```
# put these in the shell profile (or a .envrc) on the demo machine
export HF_HOME=/sgoinfre/$USER/hf_cache
export UV_CACHE_DIR=/sgoinfre/$USER/uv_cache
mkdir -p "$HF_HOME" "$UV_CACHE_DIR"

# then, from the project root:
uv sync --extra dev
uv run python -c "from transformers import AutoTokenizer; \
    AutoTokenizer.from_pretrained('Qwen/Qwen3-0.6B')"
```

Pre-downloading and offline resilience

Model downloads are the only network dependency at run time, and the defense is the worst possible moment for a registry hiccup. The warm-up sequence — run it the day before, on the exact machine:

The day-before warm-up; step 3 is the one that catches surprises

```
# 1. full model pull (weights + config + tokenizer)
uv run python - <<'PY'
from transformers import AutoModelForCausalLM, AutoTokenizer
AutoTokenizer.from_pretrained('Qwen/Qwen3-0.6B')
AutoModelForCausalLM.from_pretrained('Qwen/Qwen3-0.6B',
                                     torch_dtype='auto')

print('qwen cached')
PY

# 2. bonus models, if you demo them
uv run python - <<'PY'
from sentence_transformers import SentenceTransformer
SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
print('minilm cached')
PY

# 3. prove the cache works without network
HF_HUB_OFFLINE=1 uv run python -m src answer "What is vLLM?" --k 3
```

HF_HUB_OFFLINE=1 forces transformers to serve everything from cache and fail loudly if anything is missing — far better to see that failure alone at 6 pm than with an evaluator at 10 am. If step 3 passes, network can die during your defense and nothing changes.

Shared-machine etiquette that doubles as engineering

- torch on CPU will happily grab every core; on a shared machine set OMP_NUM_THREADS=4 (or torch.set_num_threads) for dataset-scale generation — you lose little throughput and stop starving the neighbour's session (and the evaluator's terminal).
- Run answer_dataset under nice -n 10 for long runs; index and search are too fast to matter.
- Check disk BEFORE the demo, not during: df -h on the cache volume and du -sh data/processed/. A full disk mid-index produces confusing partial-pickle errors (Volume 2's troubleshooting table, row 16).
- If the machine dual-boots or resets overnight, keep the warm-up script in the repo (scripts/warmup.sh) — re-provisioning becomes one command, and the script itself is a nice artifact to show.

Note — The subject's submission chapter says evaluators generate data and outputs during evaluation — meaning YOUR data/raw and data/processed do not travel with the repo. Budget the defense timeline accordingly: corpus placement + uv sync + warm cache + index build fit comfortably in the setup window IF the caches are warm, and painfully if they are not.

CHAPTER 21

Appendix: An Annotated Defense Transcript

A realistic forty-minute session, move by move, with the examiner's intent decoded in the margins.

What follows is a composite transcript of a strong defense of this project — condensed, but every beat is one that actually occurs. **E** is the evaluator, **S** the student. The bracketed annotations decode what is being tested, which is rarely what is literally being asked.

Minutes 0-8: setup and the automated run

E: Fresh clone. Drive.
 S: `git clone ...; cd; uv sync`. While it resolves: layout is `src/` as a module, `data/` mounted here, `Makefile` at root.
 [narrating during dead time = confidence + saved minutes]
 E: `make lint`.
 S: (green) `flake8` plus `mypy` with the subject's five flags;
`lint-strict` also passes if you want it.
 [offering the optional stricter gate unprompted: cheap credibility]
 E: Index, then docs dataset, then your *moulinette* score.
 S: (runs the four commands from history, narrates the `tqdm` bars)
 Index: 31,842 chunks in 2m10. Search: 100 questions, 1.8s.
Moulinette: valid True; R@1 .52, @3 .74, @5 .86, @10 .90.
 [knows the numbers before the screen prints them]

Minutes 8-18: exploration

E: Empty query. Then `k=0`. Then this: `search "$(cat pyproject.toml)"`
 S: (all exit clean; the last returns build-system related chunks)
 Degenerate inputs return empty or best-effort lists; the CLI
 never tracebacks - one error boundary in `__main__`.
 [the weird third input is a bluff-check: is the robustness real
 or rehearsed? real systems don't care that input is weird]
 E: Your data/processed is a pickle. Why should I trust it?
 S: Internal artifact, written and read only by my code, regenerated
 by one command; the graded wire formats are *pydantic*-validated
 JSON. If you corrupt it, `load_index` raises a clear error and
 tells you to re-index - want to see? (truncates file, shows it)
 [invited the failure demo instead of fearing it]
 E: Why are there chunks bigger than your average but none over 2000?
 S: Packing merges structural spans up to the cap; the cap itself is
 asserted at the single `Chunk` constructor. Four enforcement
 layers, one invariant - it cannot leak by construction.

Minutes 18-32: theory and code reading

E: Whiteboard BM25. What do k_1 and b do, and what did YOU use?

S: (writes the formula, sketches the saturation curve)
 k_1 caps repetition reward - gain asymptotes at k_1+1 ; b scales the saturation point by $|d|/avgdl$. Grid-searched: 1.2/0.6 for docs, defaults were within a point; numbers in the README.
 [formula + curve + OWN numbers = the complete answer shape]

E: Your analyzer emits `gpu_memory_utilization` AND its parts. Isn't that double counting?

S: Deliberate: the intact token keeps huge IDF for verbatim queries; parts catch paraphrases. For plain words no duplicate is emitted, so ordinary prose TF is undistorted. It was worth +8 docs / +15 code - biggest single lever I measured.
 [the question sounds like an objection; it is an invitation to show you understand your own best decision]

E: If I gave you a corpus of French legal PDFs tomorrow, what survives from this system?

S: The skeleton: stage contracts, harness, triage, invariants. What changes: extraction layer, analyzer (accents, no identifiers, French stoplist/stemmer), chunking on document structure, and I'd expect embeddings to matter more - legal paraphrase is worse than code paraphrase.
 [transfer question: do you own principles or a recipe?]

Minutes 32-40: recode and bonuses

E: Make evaluate print the five worst questions by `recall@5`.

S: (opens `evaluation.py` directly - no searching - adds a `sorted()` over per-question scores, prints bottom five with ids, runs it)
 [the drill from ch. 5, almost verbatim; the skill being graded is knowing WHERE the seam is in your own code]

E: You claim hybrid retrieval. Prove it beats both parents.

S: (runs the harness twice with `--mode` flags, shows the table:
 BM25 .86/.61, semantic .84/.52, hybrid .88/.63)
 Semantic loses on code - identifiers embed poorly - which is exactly why fusion wins: opposite blind spots.
 [a bonus is a measured claim, not a feature flag]

What made this defense strong

- Every number was known before the tool printed it - the lab notebook was internalized, not consulted.
- Failures were demonstrated voluntarily (the corrupt pickle) - confidence in error paths reads as deep ownership.
- Each answer had the same shape: mechanism, then decision, then measurement. Theory - choice - number. Practice that cadence.
- Dead time (installs, model load) was filled with unprompted, relevant narration - the session never stalled.
- The one unknown that came up ('does Fire support negative numbers as positional args?') got an honest 'not sure - let's test it' and a live check, not a bluff. It cost nothing.

Reread the transcript the evening before your session, and note that nothing in it requires brilliance — the case study, the gauntlet, the drills, and the two question banks in this series ARE this

transcript, decomposed into rehearsable parts. You have rehearsed all of them. Go answer some questions.

The one-page crib sheet

Everything that must be instantly recallable during the defense, on one page. Photograph it mentally; it is the whole series at maximum compression.

Item	Value
Pipeline	index -> retrieve -> augment -> generate; contracts are the pydantic models
Thresholds	recall@5: docs ≥ 0.80 , code ≥ 0.50 (moulinette verdict at $k=5$)
Match rule	verbatim file_path AND interval IoU ≥ 0.05
Hard limits	chunk span ≤ 2000 chars; index ≤ 5 min; 200 questions ≤ 90 s
BM25	$\text{idf}(t) \times \text{tf}(k1+1)/(\text{tf} + k1(1-b+b d /\text{avgdl}))$; $\text{idf} = \ln((N-\text{df}+.5)/(\text{df}+.5)+1)$; defaults 1.5/0.75
Analyzer	lowercase; split snake/camel KEEPING both forms; ~30 stopwords; same function both sides
Chunkers	md: headings then paragraphs, fences unsplit; py: AST defs, big classes per method, small spans merged; hard split + overlap as last resort
Commands	index, search, search_dataset, answer, answer_dataset, evaluate - uv run python -m src <cmd>
Generation	Qwen3-0.6B; top 3-5 snippets labelled with paths; ground + escape hatch; thinking off; ≤ 256 new tokens
Moulinette	<code>./moulinette evaluate_student_search_results <results> <truth> --k 10 --max_context_length 2000 - results FIRST</code>
Triage classes	FILE-MISS -> analyzer/coverage/hybrid; SPAN-MISS -> boundaries/size; RANK-MISS -> k1/b/boosts/dedup
Bonuses	1 MiniLM vectors, 2 RRF sum $1/(60+\text{rank})$, 3 sha256 manifest + tombstones, 4 mtime-keyed caches, 5 FastAPI over the same functions
Five decisions	what to index / where to cut / what a term is / how to score / how to prompt
Answer cadence	mechanism -> decision -> measurement. Never bluff; test live instead.

Series topic index

Where every major topic lives across the three volumes (V1 Theory, V2 Implementation, V3 this volume), for last-minute lookups:

Topic	Theory	Code	Tuning/defense
Analyzer / identifier splitting	V1 ch.8	V2 ch.4	V3 ch.3 knob 4, ch.12 #2/#6
AST chunking	V1 ch.9	V2 ch.6	V3 ch.12 #4

Topic	Theory	Code	Tuning/defense
BM25 formula and knobs	V1 ch.7	V2 ch.8	V3 ch.3 knobs 2-3, ch.19
Caching	-	-	V3 ch.9
Chat template / thinking mode	V1 ch.2, 12	V2 ch.11	V3 ch.19
Chunk size trade-off	V1 ch.9	V2 ch.5-6	V3 ch.3 knob 1, ch.11
CLI commands	-	V2 ch.9	V3 ch.5 gauntlet
Datasets anatomy	-	V2 ch.13	V3 ch.12
Edge cases / robustness	-	V2 ch.10	V3 ch.5
Embeddings / dense retrieval	V1 ch.10	-	V3 ch.6
Evaluation metric (recall@k, IoU)	V1 ch.11	V2 ch.12	V3 ch.1-2
Hybrid / RRF	V1 ch.10	-	V3 ch.7
HTTP API	-	-	V3 ch.10
Incremental indexing	-	-	V3 ch.8
Makefile / lint / mypy	-	V2 ch.1, 15	V3 ch.17
Moulinette usage	V1 ch.1	V2 ch.12	V3 ch.1
Performance budgets	V1 ch.16	V2 ch.7-8	V3 ch.4
Prompting / generation	V1 ch.12	V2 ch.11	V3 ch.13 (re-rank), ch.19
pydantic models	V1 ch.4	V2 ch.2	-
README requirements	-	-	V3 ch.15
Recode drills	-	-	V3 ch.5, ch.21
TF-IDF	V1 ch.6	-	-
Triage workflow	-	-	V3 ch.2, 11, 12
vLLM corpus map	V1 ch.13	V2 ch.3	V3 ch.12 #8
Worked examples end-to-end	V1 ch.14	V2 ch.14	V3 ch.11, 21